GATE DA 2027

Machine Learning

Supervised methods, model evaluation and unsupervised methods with the stories of Ramu and Somu

Ramu and Somu once more.
Ramu asks "but why?" about everything. **Somu** answers with a cricket coach, a fruit market, a spam folder and a tailor.

Syllabus covered (GATE DA, Section 6): *supervised:* regression (simple, multiple, ridge) and classification (logistic regression, k-nearest neighbour, naive Bayes, linear discriminant analysis, support vector machines, decision trees, multi-layer perceptron/feed-forward networks); *evaluation:* bias–variance trade-off, cross-validation (leave-one-out, k-fold); *unsupervised:* clustering (k-means, k-medoids, hierarchical single/complete/average linkage) and principal component analysis.

Every number in the worked examples was computed by code (NumPy/scikit-learn), and neural-network gradients were checked against numerical differentiation.

Contents

1 The Big Picture **3**

2 Regression **3**

2.1 Simple linear regression 3

2.2 Multiple linear regression and the normal equation 4

2.3 Ridge regression (L2 regularisation) 4

3 Classification **5**

3.1 Logistic regression 5

3.2 k-nearest neighbours (kNN) 6

3.3 Naive Bayes 6

3.4 Linear discriminant analysis (LDA) 7

3.5 Evaluating a classifier 7

4 Decision Trees **8**

5 Support Vector Machines (SVM) **9**

6 Neural Networks: Perceptron, MLP and Feed-forward Networks **10**

6.1 Perceptron, activations and architecture 10

6.2 Forward pass and backpropagation (worked, numbers verified) 11

7 Model Evaluation: Bias–Variance and Cross-validation **12**

7.1 Underfitting, overfitting and the bias–variance decomposition 12

7.2 Train/validation/test and cross-validation 13

8 Clustering **14**

8.1 k-means 14

8.2 k-medoids 14

8.3 Hierarchical clustering 15

9 Principal Component Analysis (PCA) **16**

10 One-page Cheat Sheet **17**

11 Practice Questions (with answers) **18**

1 The Big Picture

Ramu & Somu: Learning from examples

Ramu's cricket coach never gives a formula for a good cover drive. He shows many shots and says "good" or "bad" until Ramu's body learns the pattern. Somu: "A machine does the same. We give it **examples** and it finds the pattern. When every example comes with an answer (a **label**), that is **supervised learning**. When there are no answers and we only ask "what natural groups exist here?", that is **unsupervised learning**."

Vocabulary

- Data: n examples $(\boldsymbol{x}_i, y_i)$, $\boldsymbol{x}_i \in \mathbb{R}^d$ is the **feature vector**, y_i the **label/target**. Stack features in a matrix $X \in \mathbb{R}^{n\times d}$.
- **Regression**: y is a real number (price). **Classification**: y is a category (spam/ham).
- A **model** $f_{\boldsymbol{w}}$ has **parameters** $\boldsymbol{w}$ learned from data and **hyperparameters** (such as k, λ, tree depth) chosen by you.
- **Loss function** $\ell(y, \hat{y})$ measures one error; **training** minimises the average loss over the training set (empirical risk).
- **Generalisation**: how well the model does on *new* data. We estimate it on a held-out **test set**, never on the data used to train.
- **Unsupervised**: only $\boldsymbol{x}_i$ are given: clustering, dimensionality reduction (PCA).

2 Regression

2.1 Simple linear regression

Ramu & Somu: Marks versus hours

Ramu notes hours studied (x) and marks (y) for five friends, and wants a line to predict the marks of a sixth. Somu: "Among all lines, pick the one with the smallest total of squared vertical gaps (**residuals**). That is **least squares**."

Least-squares line $\hat{y} = \beta_0 + \beta_1 x$

$$\beta_1 = \frac{S_{xy}}{S_{xx}} = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2} = r\,\frac{s_y}{s_x}, \qquad \beta_0 = \bar{y} - \beta_1 \bar{x}.$$

The line always passes through $(\bar{x}, \bar{y})$; residuals $e_i = y_i - \hat{y}_i$ sum to zero.

$$\text{SSE} = \sum e_i^2, \quad \text{SST} = \sum (y_i - \bar{y})^2, \quad R^2 = 1 - \frac{\text{SSE}}{\text{SST}} \in [0, 1].$$

For *simple* regression $R^2 = r^2$ (r the correlation). R^2 is the fraction of the variance of y explained by the model.

$x = 1, 2, 3, 4, 5$ and $y = 2, 4, 5, 4, 5$

$\bar{x} = 3$, $\bar{y} = 4$, $S_{xy} = 6$, $S_{xx} = 10$. So $\beta_1 = 0.6$ and $\beta_0 = 4 - 0.6 \cdot 3 = 2.2$: $\hat{y} = 2.2 + 0.6x$.
Residuals: -0.8, 0.6, 1.0, -0.6, -0.2 (sum 0). SSE $= 2.4$, SST $= 6$, $R^2 = 1 - \frac{2.4}{6} = \mathbf{0.6}$, $r = \sqrt{0.6} = 0.7746$.
Prediction at $x = 6$: $2.2 + 3.6 = 5.8$. Errors: MSE $= 2.4/5 = 0.48$, RMSE $= 0.6928$, MAE

$= 0.64$.

2.2 Multiple linear regression and the normal equation

Matrix form

With $X \in \mathbb{R}^{n\times(d+1)}$ (first column all ones for the intercept), $\hat{y} = X\boldsymbol{w}$. Minimising $\|y - X\boldsymbol{w}\|^2$ gives the **normal equations**

$$X^\top X\,\hat{\boldsymbol{w}} = X^\top y \quad\Longrightarrow\quad \hat{\boldsymbol{w}} = (X^\top X)^{-1}X^\top y.$$

Requires $X^\top X$ invertible (columns linearly independent: no perfect multicollinearity, $n \geq d+1$). Geometrically $X\hat{\boldsymbol{w}}$ is the *projection* of y onto the column space of X, so the residual is orthogonal to every column: $X^\top(y - X\hat{\boldsymbol{w}}) = 0$.
Gradient descent alternative: $\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta\,\frac{2}{n}X^\top(X\boldsymbol{w} - y)$. Example: data $(1,2),(2,4)$, model $\hat{y} = wx$, $w_0 = 0$: gradient of MSE $= \frac{2}{2}[(-2)(1) + (-4)(2)] = -10$; with $\eta = 0.1$, $w_1 = 1.0$ (the true $w = 2$ is approached step by step).

Four points, two features

$$X = \begin{pmatrix} 1 & 1 & 2 \\ 1 & 2 & 1 \\ 1 & 3 & 4 \\ 1 & 4 & 3 \end{pmatrix},\ y = (6,6,14,13)^\top,\ X^\top X = \begin{pmatrix} 4 & 10 & 10 \\ 10 & 30 & 28 \\ 10 & 28 & 30 \end{pmatrix}.$$ Solving gives $\hat{\boldsymbol{w}} = (\frac{3}{8},\ \frac{13}{8},\ \frac{17}{8})$.

2.3 Ridge regression (L2 regularisation)

Ramu & Somu: Taming a wild line

With many features and few examples, least squares can chase noise and produce huge weights. Somu adds a *penalty on big weights*: "You may fit the data, but every unit of weight costs you λ." This shrinks the weights toward zero.

Ridge

$$\hat{\boldsymbol{w}}_{\text{ridge}} = \arg\min_{\boldsymbol{w}} \|y - X\boldsymbol{w}\|^2 + \lambda\|\boldsymbol{w}\|^2 = (X^\top X + \lambda I)^{-1}X^\top y.$$

$\lambda = 0$: ordinary least squares. $\lambda \to \infty$: weights $\to 0$. Adding λI makes $X^\top X + \lambda I$ *always invertible*, curing multicollinearity. The intercept is usually not penalised. Larger $\lambda \Rightarrow$ more bias, less variance. **Lasso** uses $\lambda\|\boldsymbol{w}\|_1$ instead and drives some weights *exactly* to zero (feature selection). Choose λ by cross-validation.

Ridge on the marks data: slope $= \dfrac{S_{xy}}{S_{xx}+\lambda}$

λ	0	1	5	100
slope β_1	0.6000	0.5455	0.4000	0.0545
intercept β_0	2.2000	2.3636	2.8000	3.8364

As λ grows the slope shrinks toward 0 and the intercept toward $\bar{y} = 4$.

Common GATE traps

(1) R^2 never decreases when you add features (use adjusted R^2 or test data). (2) Do not penalise the intercept. (3) Standardise features before ridge/lasso/kNN/SVM, since the penalty depends on scale. (4) $(X^\top X)^{-1}$ fails when features are collinear. (5) A high R^2 on training data does not mean a good model.

3 Classification

3.1 Logistic regression

Ramu & Somu: A probability, not a number

Ramu wants to predict "will it rain?" A line can output -3 or 7, which is silly for a probability. Somu squeezes any number into $(0, 1)$ with the **sigmoid** $\sigma(z) = \frac{1}{1+e^{-z}}$, and says "rain" when the probability exceeds $\frac{1}{2}$.

Model and loss

$$p(y = 1 \mid \boldsymbol{x}) = \sigma(\boldsymbol{w}^\top \boldsymbol{x} + b), \qquad \sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

Log-odds are linear: $\ln \frac{p}{1-p} = \boldsymbol{w}^\top \boldsymbol{x} + b$, so the **decision boundary** $\boldsymbol{w}^\top \boldsymbol{x} + b = 0$ (where $p = \frac{1}{2}$) is a *hyperplane*: logistic regression is a linear classifier. Fit by minimising the **cross-entropy (log) loss**

$$\ell = -\big[y \ln p + (1 - y) \ln(1 - p)\big], \qquad \frac{\partial \ell}{\partial \boldsymbol{w}} = (p - y)\, \boldsymbol{x}, \quad \frac{\partial \ell}{\partial b} = p - y.$$

The loss is convex, so gradient descent reaches the global minimum. For K classes use **softmax**: $p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$.

One example, one gradient step

$\boldsymbol{w} = (0.5, -0.25)$, $b = 0.5$, $\boldsymbol{x} = (2, 4)$, $y = 1$. Then $z = 1 - 1 + 0.5 = 0.5$, $p = \sigma(0.5) = \mathbf{0.6225}$. Loss $= -\ln 0.6225 = 0.4741$ (if the label were 0, loss $= 0.9741$). Gradient: $(p - 1)\boldsymbol{x} = (-0.755, -1.510)$, $\partial b = -0.3775$. With $\eta = 0.1$: $\boldsymbol{w} \leftarrow (0.5755, -0.0990)$, $b \leftarrow 0.5378$ and the new probability is 0.7846: the model is now more confident in the right class.
Useful values: $\sigma(0) = 0.5$, $\sigma(2) = 0.8808$, $\sigma(-2) = 0.1192$, $\sigma(1) = 0.7311$. Softmax of $(2, 1, 0.1)$ is $(0.659, 0.242, 0.099)$.

3.2 k-nearest neighbours (kNN)

Ramu & Somu: Ask the neighbours

A new family moves in. Somu: "To guess their cricket team, ask the k closest houses and go with the majority." No training at all; all work happens at prediction time (a **lazy learner**).

kNN

Classify $\boldsymbol{x}$ by the majority label among its k nearest training points (distance: Euclidean $\sqrt{\sum (x_j - z_j)^2}$, Manhattan $\sum |x_j - z_j|$, or other). For regression: average of the k neighbours' values. **Small** k: flexible, low bias, high variance (overfits; $k = 1$ gives zero training error). **Large** k: smooth, high bias (as $k = n$ it predicts the global majority). Features must be **scaled**. Prediction cost $O(nd)$ per query; it suffers in high dimensions (curse of dimensionality).

Query $(4, 4)$

Training points: A: $(1,1), (2,1), (2,3)$; B: $(6,5), (7,7), (8,6)$. Euclidean distances to $(4,4)$: 4.24, 3.61, 2.24, 2.24, 4.24, 4.47. The 3 nearest are $(2,3)$:A and $(6,5)$:B (both 2.24) and $(2,1)$:A (3.61): two A against one B, so $k = 3 \Rightarrow$ **A**. (With $k = 1$ there is a tie, which is why k is usually odd.) Manhattan distances $(6, 5, 3, 3, 6, 6)$ give the same three neighbours.

3.3 Naive Bayes

Ramu & Somu: Spam by counting words

Somu counts: of the spam emails, how many contain "free", how many "win"? For a new email he multiplies these chances, pretending the words appear *independently* once you know the class. That is "naive" but works surprisingly well.

Naive Bayes classifier

By Bayes' rule $P(C \mid \boldsymbol{x}) \propto P(C) \prod_{j=1}^{d} P(x_j \mid C)$ (conditional independence of features given the class). Predict $\hat{C} = \arg\max_C P(C) \prod_j P(x_j \mid C)$. Probabilities are estimated by counting; **Laplace smoothing** fixes the zero-frequency problem: $P(x_j = v \mid C) = \dfrac{\text{count} + 1}{N_C + V}$ (V = number of values the feature can take). For continuous features use a Gaussian per class (mean and variance). Training is one pass over the data.

Six emails

Spam: $(\text{free}, \text{win}) = (1,1), (1,0), (0,1)$; ham: $(0,0), (1,0), (0,0)$. New email $(1,1)$. Priors $\frac{1}{2}, \frac{1}{2}$.
Without smoothing: spam $= \frac{1}{2} \cdot \frac{2}{3} \cdot \frac{2}{3} = 0.2222$; ham $= \frac{1}{2} \cdot \frac{1}{3} \cdot 0 = 0$ (no ham email had "win": a single zero kills the product).
Laplace ($V = 2$): spam $= \frac{1}{2} \cdot \frac{3}{5} \cdot \frac{3}{5} = 0.18$; ham $= \frac{1}{2} \cdot \frac{2}{5} \cdot \frac{1}{5} = 0.04$. Posterior $P(\text{spam}) = \frac{0.18}{0.18 + 0.04} =$ **0.818**.

3.4 Linear discriminant analysis (LDA)

LDA

Assume each class is Gaussian with its own mean μ_k but a **shared covariance** Σ. The Bayes-optimal rule then has linear discriminants

$$\delta_k(\boldsymbol{x}) = \boldsymbol{x}^\top \Sigma^{-1}\mu_k - \tfrac{1}{2}\mu_k^\top \Sigma^{-1}\mu_k + \ln \pi_k, \qquad \hat{C} = \arg\max_k \delta_k(\boldsymbol{x}),$$

with priors π_k. The boundary is linear. (With *different* covariances you get quadratic boundaries: QDA.) **Fisher's view:** project onto the direction $\boldsymbol{w} \propto \Sigma^{-1}(\mu_1 - \mu_2)$ that maximises between-class separation relative to within-class scatter.
One dimension, two classes: boundary $x^* = \frac{\mu_1+\mu_2}{2} - \frac{\sigma^2 \ln(\pi_1/\pi_2)}{\mu_1-\mu_2}$.

$\mu_1 = 2,\ \mu_2 = 6,\ \sigma^2 = 4$

Equal priors: $x^* = \frac{2+6}{2} = 4$ (the midpoint). Priors $\pi_1 = 0.7, \pi_2 = 0.3$: $x^* = 4 - \frac{4\ln(7/3)}{-4} = 4 + 0.847 = \mathbf{4.847}$: the boundary moves *toward* the rarer class 2, because class 1 is a priori more likely.

3.5 Evaluating a classifier

Confusion matrix metrics

	predicted +	predicted −
actual +	TP	FN
actual −	FP	TN

Accuracy $= \frac{TP+TN}{N}$; precision $= \frac{TP}{TP+FP}$; recall (sensitivity, TPR) $= \frac{TP}{TP+FN}$; specificity $= \frac{TN}{TN+FP}$; FPR $= 1-$specificity; $F_1 = \frac{2PR}{P+R}$ (harmonic mean). Example: $TP = 40, FP = 10, FN = 20, TN = 30$: accuracy 0.70, precision 0.80, recall 0.667, specificity 0.75, $F_1 = 0.727$. The **ROC curve** plots TPR against FPR as the threshold varies; **AUC** (area under it) is 0.5 for random guessing and 1 for perfect ranking. With imbalanced classes accuracy misleads: a model that always says "no" on a 99%-negative dataset gets 99% accuracy and 0 recall.

Common GATE traps

(1) Logistic "regression" is a classifier. (2) Naive Bayes needs smoothing for unseen feature values. (3) kNN has no training phase but costly prediction. (4) LDA assumes equal covariances; QDA does not. (5) Precision and recall trade off as the threshold moves. (6) Standardise features for distance-based methods.

4 Decision Trees

Ramu & Somu: Twenty questions

Ramu thinks of an animal and Somu asks yes/no questions. A good player first asks the question that cuts the possibilities most evenly ("Does it live in water?"), not a silly one ("Is it a dog called Moti?"). A **decision tree** is a flowchart of such questions: each internal node tests a feature, each branch is an answer, each leaf gives a class. Learning means choosing, node by node, the *most informative* question.

Impurity measures

For a node with class proportions $p_1, \ldots, p_K$:

- **Entropy** $H = -\sum_k p_k \log_2 p_k$ (bits). $H = 0$ for a pure node; $H = \log_2 K$ at most (for two classes, 1 bit at 50/50). Example: $(1, 3) \to 0.8113$.
- **Gini index** $G = 1 - \sum_k p_k^2$ (two classes: max 0.5).
- **Information gain** of splitting on attribute A: $\text{IG} = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)$ (ID3 picks the maximum).
- **Gain ratio** (C4.5) = IG/SplitInfo, with SplitInfo $= -\sum \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}$; it penalises attributes with many values. CART uses Gini (binary splits).
- **Regression trees**: leaf predicts the mean; split to maximise reduction in variance (SSE).

Classic weather data: 14 days, 9 "play" and 5 "no play"; split on Outlook

Outlook: sunny $(2, 3)$, overcast $(4, 0)$, rain $(3, 2)$ (yes, no).

- $H(S) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = \mathbf{0.940}$; $\text{Gini}(S) = 1 - (\frac{9}{14})^2 - (\frac{5}{14})^2 = 0.459$.
- $H(\text{sunny}) = 0.971$, $H(\text{overcast}) = 0$, $H(\text{rain}) = 0.971$.
- Remainder $= \frac{5}{14}(0.971) + \frac{4}{14}(0) + \frac{5}{14}(0.971) = 0.694$, so $\text{IG}(\text{Outlook}) = 0.940 - 0.694 = \mathbf{0.247}$.
- Weighted Gini after the split $= \frac{5}{14}(0.48) + 0 + \frac{5}{14}(0.48) = 0.343$ (down from 0.459).
- SplitInfo $= 1.577$, gain ratio $= 0.247/1.577 = 0.156$.

The overcast branch is already pure: it becomes a leaf "play".

Growing and controlling a tree

Greedy, top-down, recursive: pick the best split, partition, recurse; stop when a node is pure, no features remain, or a depth/size limit is reached. A fully grown tree memorises the training set (**overfitting**: low bias, high variance). Remedies: **pre-pruning** (max depth, minimum samples, minimum gain), **post-pruning** (grow, then cut back using validation data), and ensembles (random forests average many trees; boosting adds trees sequentially). Trees handle mixed feature types, need no scaling, are easy to interpret, but are unstable (small data changes change the tree) and produce axis-parallel boundaries.

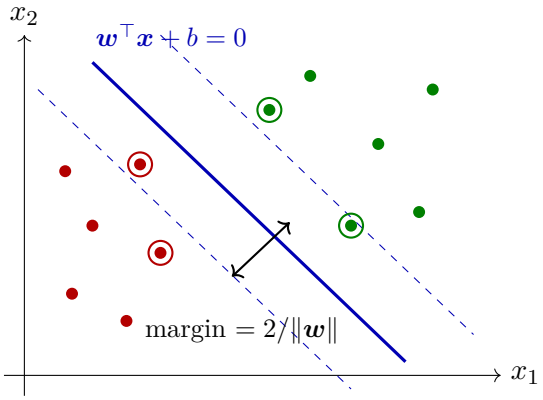
Common GATE traps

(1) Information gain favours attributes with many values (hence gain ratio). (2) Entropy uses $\log_2$ (bits); with natural logs the values change by a constant factor. (3) $0 \log 0 = 0$. (4) A tree can always reach 100% training accuracy if no two identical inputs have different labels. (5) Do not confuse the weighted impurity *after* the split with the gain.

5 Support Vector Machines (SVM)

Ramu & Somu: The widest road

Two villages, red on the left and blue on the right. Somu must build a straight road between them. "Many roads separate the villages. The safest is the *widest*, the one that keeps the greatest clearance from the nearest house on each side." The nearest houses are the **support vectors**: they alone hold the road in place. Move any other house and the road stays; move a support vector and the road shifts.



Hard-margin SVM

Labels $y_i \in \{-1, +1\}$. Require every point on the correct side with clearance:

$$\min_{\boldsymbol{w},b} \ \tfrac{1}{2}\|\boldsymbol{w}\|^2 \quad \text{subject to} \quad y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) \geq 1 \ \ \forall i.$$

The two margin lines are $\boldsymbol{w}^\top \boldsymbol{x} + b = \pm 1$; the distance between them (the **margin**) is $\dfrac{2}{\|\boldsymbol{w}\|}$, so maximising the margin = minimising $\|\boldsymbol{w}\|^2$. Points with $y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) = 1$ are the **support vectors**. Predict $\text{sign}(\boldsymbol{w}^\top \boldsymbol{x} + b)$.
Dual form: $\boldsymbol{w} = \sum_i \alpha_i y_i \boldsymbol{x}_i$ with $\alpha_i \geq 0$, $\sum_i \alpha_i y_i = 0$, and $\alpha_i > 0$ *only* for support vectors. Predictions use only dot products $\boldsymbol{x}_i^\top \boldsymbol{x}$, which makes the kernel trick possible.

Soft margin and kernels

Real data overlap, so allow **slack** $\xi_i \geq 0$: minimise $\frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_i \xi_i$ subject to $y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) \geq 1 - \xi_i$. Equivalent to the **hinge loss** $\max(0,\, 1 - y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b))$ plus an L2 penalty. C **large**: few violations, narrow margin (low bias, high variance, risk of overfitting); C **small**: wide margin, more violations (more bias, less variance).
Kernel trick: replace $\boldsymbol{x}_i^\top \boldsymbol{x}_j$ by $K(\boldsymbol{x}_i, \boldsymbol{x}_j) = \phi(\boldsymbol{x}_i)^\top \phi(\boldsymbol{x}_j)$ to get a non-linear boundary without computing ϕ. Common kernels: linear $\boldsymbol{x}^\top \boldsymbol{z}$; polynomial $(\boldsymbol{x}^\top \boldsymbol{z} + c)^d$; RBF/Gaussian $\exp(-\gamma\|\boldsymbol{x} - \boldsymbol{z}\|^2)$ (small γ: smooth; large γ: wiggly, overfits).

Two small problems, solved by an SVM solver

One dimension: negatives at $x = 1, 2$, positives at $x = 4, 5$. Support vectors $x = 2$ and $x = 4$; $w = 1$, $b = -3$ (boundary $x = 3$), margin $\frac{2}{|w|} = 2$.
Two dimensions: negatives $(1,1), (2,1), (1,2)$; positives $(4,4), (5,4), (4,5)$. Support vectors $(2,1), (1,2), (4,4)$; $\boldsymbol{w} = (0.4, 0.4)$, $b = -2.2$, so the boundary is $x_1 + x_2 = 5.5$ and the margin

is $\frac{2}{0.4\sqrt{2}} = \frac{5}{\sqrt{2}} = 3.54$. Check: $0.4(2+1) - 2.2 = -1$ and $0.4(4+4) - 2.2 = +1$. Multipliers $\alpha = (0.08, 0.08, 0.16)$, which satisfy $\sum \alpha_i y_i = -0.08 - 0.08 + 0.16 = 0$.
XOR points $(0,0), (1,1) \to -1$; $(0,1), (1,0) \to +1$: no line separates them (a linear SVM scores 50%), but a polynomial kernel of degree 2 separates them perfectly (100%).

Common GATE traps

(1) The margin is $2/\|\boldsymbol{w}\|$, not $1/\|\boldsymbol{w}\|$. (2) Only support vectors matter; deleting a non-support vector does not change the model. (3) Larger C means a *harder* margin. (4) Scale features before using RBF or any distance-based kernel. (5) SVM outputs a class, not a probability, unless calibrated.

6 Neural Networks: Perceptron, MLP and Feed-forward Networks

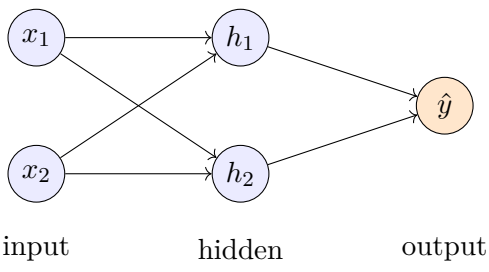
Ramu & Somu: A committee of simple voters

Ramu's college chooses a topper using a committee. Each member looks at the marks, weighs them by his own taste, and gives a score. A second layer of members then combines those scores. No single member is clever, but the layered committee can learn very complicated rules. Each member is a **neuron**: it computes activation$(\boldsymbol{w}^\top \boldsymbol{x} + b)$. Layers of neurons form a **multi-layer perceptron (MLP)**, a **feed-forward network** because information flows only forward.

6.1 Perceptron, activations and architecture

Building blocks

- **Perceptron:** $\hat{y} = \mathbb{1}[\boldsymbol{w}^\top \boldsymbol{x} + b > 0]$. Update on a mistake: $\boldsymbol{w} \leftarrow \boldsymbol{w} + \eta\,(y - \hat{y})\,\boldsymbol{x}$. It converges *if the data are linearly separable*; a single perceptron cannot learn XOR.
- **Layer:** $\boldsymbol{h} = \phi(W\boldsymbol{x} + \mathbf{b})$ with weight matrix W (rows are neurons). Network: $\hat{y} = \phi_L(W_L \cdots \phi_1(W_1\boldsymbol{x} + \mathbf{b}_1) \cdots)$.
- **Activations:** sigmoid $\sigma(z)$ (range $(0,1)$, $\sigma' = \sigma(1-\sigma) \le \frac{1}{4}$); tanh (range $(-1,1)$, $\tanh 1 = 0.7616$); ReLU $\max(0, z)$ (cheap, no saturation for $z > 0$); softmax for multi-class outputs. **Without a non-linear activation** a stack of layers collapses to a single linear map.
- **Parameter count:** a layer with n_{in} inputs and n_{out} neurons has $n_{in}n_{out} + n_{out}$ parameters. Network $4{-}5{-}3$: $(4 \cdot 5 + 5) + (5 \cdot 3 + 3) = \mathbf{43}$; $784{-}128{-}10$: 101 770; $10{-}8{-}8{-}2$: 178.
- **Universal approximation:** one hidden layer with enough neurons and a non-linear activation can approximate any continuous function on a compact set (to any accuracy).
- **Loss:** squared error for regression, cross-entropy (with softmax/sigmoid output) for classification.



XOR with two ReLU hidden units

Hidden: $h_1 = \mathrm{ReLU}(x_1 + x_2)$, $h_2 = \mathrm{ReLU}(x_1 + x_2 - 1)$; output $\hat{y} = h_1 - 2h_2$. Check: $(0,0) \to 0$; $(0,1) \to h = (1,0) \to 1$; $(1,0) \to 1$; $(1,1) \to h = (2,1) \to 2 - 2 = 0$. XOR solved, which no single perceptron can do.

6.2 Forward pass and backpropagation (worked, numbers verified)

Ramu & Somu: Passing the blame backwards

After the committee gives a wrong answer, Somu traces *who contributed how much to the error*, starting from the last layer and moving backwards, using the chain rule. Each weight is nudged in the direction that reduces the error.

Backpropagation recipe (sigmoid units, squared loss $L = \frac{1}{2}(\hat{y} - t)^2$)

Forward: $\mathbf{z}_1 = W_1 \boldsymbol{x} + \mathbf{b}_1$, $\boldsymbol{h} = \sigma(\mathbf{z}_1)$, $z_2 = \mathbf{w}_2^\top \boldsymbol{h} + b_2$, $\hat{y} = \sigma(z_2)$.
Backward: $\delta_2 = (\hat{y} - t)\,\hat{y}(1 - \hat{y})$; $\partial L/\partial \mathbf{w}_2 = \delta_2\, \boldsymbol{h}$, $\partial L/\partial b_2 = \delta_2$;
$\boldsymbol{\delta}_1 = (\mathbf{w}_2\, \delta_2) \odot \boldsymbol{h} \odot (1 - \boldsymbol{h})$; $\partial L/\partial W_1 = \boldsymbol{\delta}_1 \boldsymbol{x}^\top$, $\partial L/\partial \mathbf{b}_1 = \boldsymbol{\delta}_1$.
Update: $\theta \leftarrow \theta - \eta\, \partial L/\partial \theta$ (stochastic gradient descent on mini-batches).

A $2{-}2{-}1$ network

Input $\boldsymbol{x} = (1, 0)$, target $t = 1$; $W_1 = \begin{pmatrix} 0.5 & -0.5 \\ 0.3 & 0.8 \end{pmatrix}$, $\mathbf{b}_1 = (0.1, -0.2)$, $\mathbf{w}_2 = (0.6, -0.4)$, $b_2 = 0.05$.

- Forward: $\mathbf{z}_1 = (0.6, 0.1)$, $\boldsymbol{h} = (0.6457, 0.5250)$, $z_2 = 0.2274$, $\hat{y} = 0.5566$, loss $= \frac{1}{2}(0.5566 - 1)^2 = 0.0983$.
- Backward: $\delta_2 = (0.5566 - 1)(0.5566)(0.4434) = -0.1094$; $\partial L/\partial \mathbf{w}_2 = (-0.0707, -0.0574)$; $\boldsymbol{\delta}_1 = (-0.0150,\ 0.0109)$; $\partial L/\partial W_1 = \boldsymbol{\delta}_1 \boldsymbol{x}^\top$ has first column $(-0.0150, 0.0109)$ and second column 0 (because $x_2 = 0$).
- Check: a numerical derivative of the loss with respect to $W_1[1,1]$ gives -0.015021, equal to the backprop value.
- After one update with $\eta = 0.5$: $\hat{y} = 0.5801$ and loss $= 0.0882$ (down from 0.0983).

Common GATE traps

(1) Sigmoid/tanh saturate, so gradients can vanish in deep networks (ReLU helps). (2) Never initialise all weights equally (symmetry: every neuron in a layer would learn the same thing). (3) Biases are parameters too. (4) Gradient descent finds a *local* minimum; the neural-network loss is non-convex. (5) A deeper network overfits easily: use regularisation (weight decay, dropout, early stopping).

7 Model Evaluation: Bias–Variance and Cross-validation

Ramu & Somu: Two bad students

Before an exam Ramu studies only the chapter titles: he scores badly on practice tests *and* on the real exam (**underfitting**). Somu memorises last year's paper word for word: he scores 100% on that paper but fails when the questions change slightly (**overfitting**). "A good student understands the pattern," says Somu, "neither too simple nor too memorised. And the practice must come from questions you have *not already seen*."

7.1 Underfitting, overfitting and the bias–variance decomposition

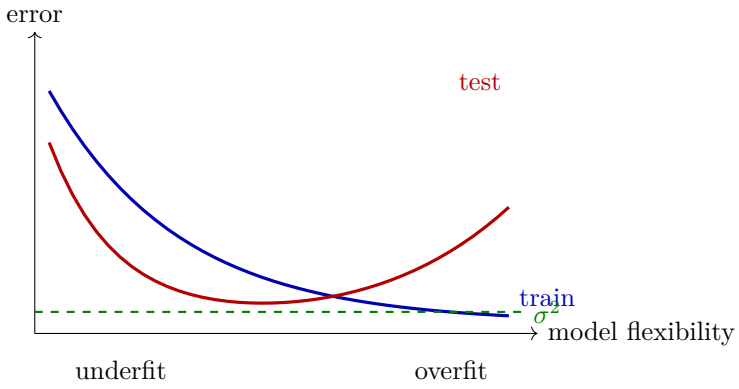
Decomposition of expected squared error

Suppose $y = f(x) + \varepsilon$ with noise $\mathbb{E}[\varepsilon] = 0$, $\mathrm{Var}(\varepsilon) = \sigma^2$, and let $\hat{f}$ be a model trained on a random training set. At a point x:

$$\mathbb{E}\big[(y - \hat{f}(x))^2\big] = \underbrace{\big(\mathbb{E}[\hat{f}(x)] - f(x)\big)^2}_{\text{bias}^2} + \underbrace{\mathrm{Var}\big(\hat{f}(x)\big)}_{\text{variance}} + \underbrace{\sigma^2}_{\text{irreducible noise}}.$$

- **Bias**: error from wrong assumptions (a line fitted to a curve). Simple models: high bias.
- **Variance**: how much $\hat{f}$ changes if the training set changes. Flexible models: high variance.
- **Noise** σ^2 cannot be reduced by any model.

Training error falls steadily as flexibility grows; *test* error falls, then rises (U-shape). The best model sits at the bottom of the U.



Polynomial fits to $y = \sin 2\pi x +$ noise ($\sigma = 0.25$), 12 training points, simulated

degree	train MSE	test MSE	diagnosis
1	0.145	0.544	underfit: high bias (both errors high)
3	0.026	0.101	good fit
9	0.006	1.427	overfit: high variance (train tiny, test huge)

The noise floor is $\sigma^2 = 0.0625$, so a test MSE of about 0.10 for degree 3 is close to the best possible. The degree-9 curve wiggles through every training point and is wildly wrong between them.

How the knobs move bias and variance

Change	Bias	Variance
more model flexibility (polynomial degree, tree depth, network size)	↓	↑
larger ridge/lasso λ	↑	↓
larger k in kNN	↑	↓
larger SVM C	↓	↑
more training data	unchanged	↓
ensembling (bagging)	about same	↓

Diagnosis: high train error and high test error ⇒ bias problem (use a richer model). Low train error but high test error ⇒ variance problem (regularise, simplify, or get more data).

7.2 Train/validation/test and cross-validation

Ramu & Somu: Honest practice tests

Somu must pick the best k for kNN. If he checks every k on the same data he will end up fooling himself. So he sets aside a **validation set** for choosing hyperparameters and a final **test set** touched only once at the end. But when data are scarce, wasting a big chunk on validation hurts. **Cross-validation** lets every example serve as validation data exactly once.

k-fold cross-validation

Split the data randomly into k equal **folds**. For $i = 1..k$: train on the other $k-1$ folds, evaluate on fold i. Report the **average** of the k scores. With $n = 100$ and $k = 5$: each model trains on 80 and is validated on 20; 5 models are trained. Typical $k = 5$ or 10. **Stratified** k-fold keeps the class proportions in every fold.

Leave-one-out cross-validation (LOOCV)

The special case $k = n$: each fold is a single example; train n models, each on $n-1$ points. Nearly unbiased estimate of generalisation error, but expensive (n fits) and the estimate can have high variance. For linear regression there is a shortcut (no refitting): LOO error$_i = \frac{e_i}{1-h_{ii}}$ where h_{ii} is the leverage.

LOOCV by hand

Data $y = \{2, 4, 6, 8\}$; the "model" predicts the mean of the training points. Leave out 2: predict $\frac{4+6+8}{3} = 6$, error $(2-6)^2 = 16$. Leave out 4: predict 5.333, error 1.778. Leave out 6: predict 4.667, error 1.778. Leave out 8: predict 4, error 16. LOOCV MSE $= \frac{16+1.778+1.778+16}{4} = \mathbf{8.889}$.

Rules of the road

(1) Fit *everything* (scaling, feature selection, imputation) inside each training fold; otherwise information leaks from the validation fold. (2) Use the test set once. (3) Larger k: less biased estimate, more computation, higher variance (LOOCV is the extreme). (4) **Nested** cross-validation (inner loop for hyperparameters, outer loop for the estimate) gives an honest score. (5) For time series use forward-chaining splits, not random folds.

Common GATE traps

(1) Training error is a poor estimate of test error. (2) LOOCV trains n models, k-fold trains k. (3) CV estimates performance; the final model is then retrained on all the data. (4) More data reduces variance, not bias. (5) Do not tune hyperparameters on the test set.

8 Clustering

Ramu & Somu: Sorting a fruit market

In a market, Ramu sees a heap of mixed fruit, with no labels. Somu groups them by looking: round and red together, long and yellow together. Nobody told him the names; he used *similarity*. That is **clustering**: find groups so that points in a group are close to each other and far from other groups.

8.1 k-means

Lloyd's algorithm

Choose k. Initialise k centroids. Repeat until nothing changes: **(1) Assignment:** put each point in the cluster of its nearest centroid. **(2) Update:** move each centroid to the mean of its cluster. It minimises the **within-cluster sum of squares** $\text{WCSS} = \sum_{j=1}^{k} \sum_{\boldsymbol{x} \in C_j} \|\boldsymbol{x} - \mu_j\|^2$; each step cannot increase WCSS, so the algorithm always **converges**, but only to a *local* minimum that depends on the initial centroids (use k**-means++** or several restarts). Cost per iteration $O(nkd)$.
Weaknesses: must choose k (elbow method on WCSS, silhouette score), assumes roughly round, similar-size clusters, sensitive to outliers and scale; the mean is not a data point.

1-D data $\{2, 3, 4, 10, 11, 12\}$, $k = 2$, initial centroids 3 and 4

iteration	centroids	assignment	new centroids
1	(3, 4)	$\{2,3\} \to c_1$; $\{4,10,11,12\} \to c_2$	(2.5, 9.25)
2	(2.5, 9.25)	$\{2,3,4\} \to c_1$; $\{10,11,12\} \to c_2$	(3, 11)
3	(3, 11)	same assignment	(3, 11): **converged**

Final clusters $\{2, 3, 4\}$ and $\{10, 11, 12\}$; WCSS $= (1 + 0 + 1) + (1 + 0 + 1) = \mathbf{4}$. Note that the point 4 is nearer to $c_2 = 4$ at first, then switches to cluster 1 when the centroids move.

8.2 k-medoids

k-medoids (PAM)

Like k-means, but each cluster centre is an *actual data point* (a **medoid**), and the cost is the sum of dissimilarities (any distance, often Manhattan). Swap a medoid with a non-medoid if it lowers the total cost. More robust to outliers and works with arbitrary distance matrices, but slower ($O(k(n-k)^2)$ per iteration).

Points $\{1, 2, 3, 10, 11\}$, $k = 2$

Medoids $\{2, 10\}$: cost $|1-2|+0+|3-2|+0+|11-10| = \mathbf{3}$. Medoids $\{1, 11\}$: $0+1+2+1+0 = 4$. The first choice is better, and the centres 2 and 10 are real data points (a mean such as 2.0 and 10.5 would not be).

8.3 Hierarchical clustering

Ramu & Somu: A family tree of points

Somu doesn't fix the number of groups. He starts with every point alone and keeps merging the two closest groups, recording the distance at each merge. The record is a **dendrogram**; cutting it at any height gives a clustering.

Agglomerative (bottom-up) vs divisive (top-down)

Agglomerative: start with n singletons, merge the closest pair of clusters repeatedly ($n-1$ merges) until one cluster remains. **Divisive**: start with one cluster and split recursively. The distance between clusters A and B defines the **linkage**:

- **Single**: $\min_{a\in A, b\in B} d(a, b)$ (nearest pair). Can produce long "chains".
- **Complete**: $\max_{a\in A, b\in B} d(a, b)$ (farthest pair). Compact clusters.
- **Average**: mean of all $|A||B|$ pairwise distances.
- (Centroid and Ward linkage also exist.)

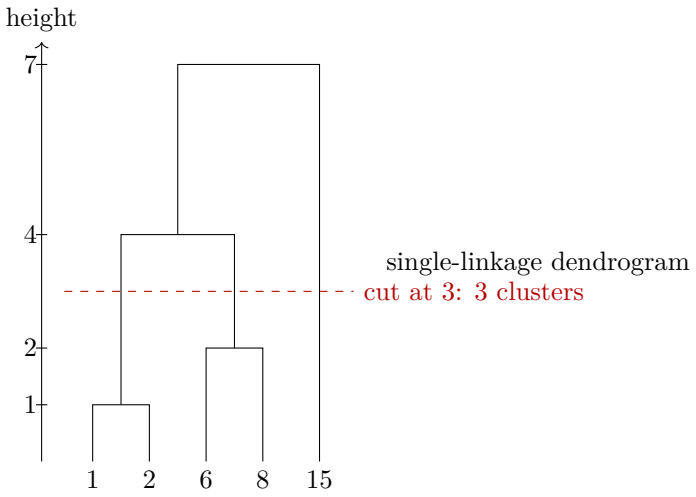
Naive agglomerative clustering costs $O(n^3)$ time (with a priority queue $O(n^2 \log n)$) and $O(n^2)$ memory.

1-D points $1, 2, 6, 8, 15$

First merges: $\{1, 2\}$ at distance 1, then $\{6, 8\}$ at 2 (all linkages agree). Then $\{1, 2\}$ with $\{6, 8\}$: single $= \min(5, 7, 4, 6) = \mathbf{4}$; complete $= \max = \mathbf{7}$; average $= \frac{5+7+4+6}{4} = \mathbf{5.5}$. The point 15 joins last:

linkage	merge 1	merge 2	merge 3	merge 4
single	1	2	4	7
complete	1	2	7	14
average	1	2	5.5	10.75

(Single: 15 to the nearest point 8 is 7. Complete: 15 to the farthest point 1 is 14. Average: $\frac{14+13+9+7}{4} = 10.75$.)



Common GATE traps

(1) k-means converges but not necessarily to the global optimum. (2) k-means uses *means*; k-medoids uses real points. (3) Single linkage uses the *minimum* distance, complete the *maximum*. (4) Hierarchical clustering is deterministic (no random start) but cannot undo a merge. (5) Always standardise features before clustering. (6) Clustering has no "accuracy" without labels: use WCSS, silhouette, or domain judgement.

9 Principal Component Analysis (PCA)

Ramu & Somu: Finding the best camera angle

Somu photographs a long, thin table. From the end, the photo shows a tiny dot (almost all information lost). From above, the full length shows up. **PCA** finds the best viewpoint: the direction along which the data *vary the most*, then the next best direction at right angles, and so on. Keep the first few directions and you compress the data while losing as little variation as possible.

PCA algorithm

1. **Centre** the data (subtract the mean of each feature; standardise too if the scales differ).
2. Compute the **covariance matrix** $C = \frac{1}{n-1} X_c^\top X_c$ (a $d \times d$ symmetric positive semi-definite matrix).
3. Find its **eigenvalues** $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d \geq 0$ and orthonormal **eigenvectors** $\mathbf{v}_1, \ldots, \mathbf{v}_d$. The eigenvectors are the **principal components (PC)** directions; λ_j is the variance along $\mathbf{v}_j$.
4. Keep the top m; project: $Z = X_c V_m$ (the scores).

Facts: $\sum_j \lambda_j = \mathrm{tr}(C) =$ total variance. Explained variance ratio of $\mathrm{PC}_j = \lambda_j / \sum \lambda$. Components are orthogonal and the projected scores are *uncorrelated*. PCA minimises the squared reconstruction error among all m-dimensional linear projections. Via the **SVD** $X_c = U \Sigma V^\top$: the columns of V are the PCs and $\lambda_j = \sigma_j^2/(n-1)$. PCA is unsupervised and linear; it does not use labels.

A 2×2 covariance matrix

$C = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$: characteristic equation $(2-\lambda)^2 - 1 = 0 \Rightarrow \lambda = 3, 1$. Eigenvectors: $\lambda = 3 \to \frac{1}{\sqrt{2}}(1,1)$ and $\lambda = 1 \to \frac{1}{\sqrt{2}}(1,-1)$. Total variance $= 4 = 3 + 1$; PC1 explains $\frac{3}{4} = \mathbf{75}\%$ and PC2 25%. Keeping only PC1 means projecting each point on the diagonal line $y = x$.

Ten 2-D points, computed by software

$x = 2.5, 0.5, 2.2, 1.9, 3.1, 2.3, 2.0, 1.0, 1.5, 1.1$ and $y = 2.4, 0.7, 2.9, 2.2, 3.0, 2.7, 1.6, 1.1, 1.6, 0.9$. Means (1.81, 1.91). Covariance $C = \begin{pmatrix} 0.6166 & 0.6154 \\ 0.6154 & 0.7166 \end{pmatrix}$. Eigenvalues $\lambda_1 = 1.284$, $\lambda_2 = 0.0491$. First component (unit vector) $\mathbf{v}_1 = (0.678, 0.735)$. Explained variance: PC1 $= \frac{1.284}{1.333} = \mathbf{96.3}\%$, PC2 $= 3.7\%$. The SVD singular values are 3.399 and 0.665 and $\frac{3.399^2}{9} = 1.284$ confirms the link. Reducing from 2 numbers to 1 keeps 96% of the variance. (The sign of a PC is arbitrary.)

How many components?

Keep enough to reach a target cumulative explained variance (for example 90–95%), or look for the **elbow** in the scree plot of eigenvalues, or use cross-validation on a downstream task. With d features and n samples there are at most $\min(n-1, d)$ non-zero eigenvalues.

Common GATE traps

(1) Centre the data first; standardise when features have different units. (2) PCA directions maximise *variance*, which need not be the directions that separate classes (LDA does that). (3) Eigenvalues are variances, not standard deviations. (4) PCs are orthogonal. (5) The sum of the eigenvalues equals the trace of C. (6) Using fewer components always loses some variance unless the rest are exactly zero.

10 One-page Cheat Sheet

Supervised methods

Linear regression: $\beta_1 = S_{xy}/S_{xx}$, $\beta_0 = \bar{y} - \beta_1 \bar{x}$, $R^2 = 1 - \text{SSE/SST}$; normal equation $\hat{\boldsymbol{w}} = (X^\top X)^{-1} X^\top y$; ridge $(X^\top X + \lambda I)^{-1} X^\top y$. Logistic: $p = \sigma(\boldsymbol{w}^\top \boldsymbol{x} + b)$, gradient $(p - y)\boldsymbol{x}$, linear boundary. kNN: small k high variance. Naive Bayes: $\arg\max P(C) \prod P(x_j|C)$ with Laplace smoothing. LDA: shared Σ, linear boundary. Tree: $H = -\sum p \log_2 p$, IG $= H(S) - \sum \frac{|S_v|}{|S|} H(S_v)$, Gini $= 1 - \sum p^2$. SVM: margin $2/\|\boldsymbol{w}\|$, support vectors, kernels; large $C \Rightarrow$ narrow margin. NN: $\hat{y} = \phi(W\boldsymbol{x} + b)$ layer by layer; params $= n_{in} n_{out} + n_{out}$; backprop uses the chain rule.

Evaluation

error $=$ bias^2 $+$ variance $+ \sigma^2$. Underfit: both errors high. Overfit: train low, test high. k-fold: k fits, train on $\frac{k-1}{k}$ of data. LOOCV: $k = n$. Precision $= \frac{TP}{TP+FP}$, recall $= \frac{TP}{TP+FN}$, F_1 harmonic mean.

Unsupervised

k-means: assign to nearest centroid, update to cluster mean; minimises WCSS; local optimum. k-medoids: centres are data points. Linkage: single = min, complete = max, average = mean pairwise distance. PCA: eigenvectors of the covariance matrix; variance ratio $\lambda_j / \sum \lambda$; $\sum \lambda = \operatorname{tr} C$; $\lambda_j = \sigma_j^2/(n-1)$.

11 Practice Questions (with answers)

Try each one yourself first. All numeric answers were computed by code.

1. For $x = 1, 2, 3, 4, 5$ and $y = 2, 4, 5, 4, 5$, find the least-squares line and R^2. **Answer:** $\hat{y} = 2.2 + 0.6x$; $R^2 = 0.6$.
2. Prediction of that line at $x = 6$? **Answer:** 5.8.
3. Slope after ridge regularisation with $\lambda = 5$ on the same data (intercept unpenalised)? **Answer:** $\dfrac{6}{10+5} = 0.4$.
4. As $\lambda \to \infty$ in ridge regression, the weights tend to? **Answer:** 0 (bias rises, variance falls).
5. Logistic regression with $\boldsymbol{w} = (0.5, -0.25)$, $b = 0.5$, $\boldsymbol{x} = (2, 4)$: probability of class 1? **Answer:** $\sigma(0.5) = 0.6225$.
6. Cross-entropy loss of that prediction if the true label is 1? **Answer:** $-\ln 0.6225 = 0.474$.
7. Is the decision boundary of logistic regression linear? **Answer: Yes** ($\boldsymbol{w}^\top \boldsymbol{x} + b = 0$).
8. kNN with $k = 1$: training error? **Answer:** 0 (each point is its own nearest neighbour); high variance.
9. Naive Bayes: spam score $0.5 \cdot \frac{3}{5} \cdot \frac{3}{5}$ and ham score $0.5 \cdot \frac{2}{5} \cdot \frac{1}{5}$ (Laplace). Posterior of spam? **Answer:** $0.18/0.22 = 0.818$.
10. LDA with $\mu_1 = 2, \mu_2 = 6, \sigma^2 = 4$, equal priors: decision boundary? **Answer:** $x = 4$.
11. Entropy of a node with class counts $(1, 3)$? **Answer:** 0.8113 **bits.**
12. Entropy of a pure node? **Answer:** 0.
13. S has 9 positive and 5 negative examples. $H(S)$? **Answer:** 0.940.
14. Information gain of Outlook with branches $(2, 3), (4, 0), (3, 2)$? **Answer:** 0.247.
15. Gini index of $(9, 5)$? **Answer:** 0.459.
16. SVM with $\boldsymbol{w} = (0.4, 0.4)$: margin width? **Answer:** $2/\|\boldsymbol{w}\| = 2/0.566 = 3.54$.
17. Which training points determine an SVM boundary? **Answer: only the support vectors.**
18. Can a linear SVM separate XOR? A degree-2 polynomial kernel? **Answer: No; yes.**
19. Number of parameters in a $4-5-3$ feed-forward network (with biases)? **Answer:** 43.
20. A single perceptron can learn which of AND, OR, XOR? **Answer: AND and OR, not XOR.**
21. Maximum value of $\sigma'(z)$? **Answer:** 0.25 **(at** $z = 0$**).**
22. Softmax of $(2, 1, 0.1)$: largest probability? **Answer:** 0.659.

23. Network with a linear activation in every layer is equivalent to? **Answer: a single linear model.**

24. Train error 0.01, test error 0.90. Diagnosis? **Answer: overfitting (high variance).**

25. Both train and test errors are high. Diagnosis? **Answer: underfitting (high bias).**

26. 5-fold CV on 100 examples: training-set size per fold and number of models fitted? **Answer:** 80 **examples;** 5 **models.**

27. LOOCV on n examples: number of models trained? **Answer:** n**.**

28. LOOCV MSE for $y = \{2, 4, 6, 8\}$ using the mean of the rest as the prediction? **Answer:** 8.889**.**

29. Precision, recall and F_1 for $TP = 40, FP = 10, FN = 20, TN = 30$? **Answer:** 0.80, 0.667, 0.727**.**

30. k-means on $\{2, 3, 4, 10, 11, 12\}$ from centroids 3 and 4: final centroids and WCSS? **Answer:** 3 **and** 11**; WCSS** $= 4$**.**

31. k-medoids cost of medoids $\{2, 10\}$ for points $\{1, 2, 3, 10, 11\}$ (Manhattan)? **Answer:** 3**.**

32. Single, complete and average linkage distances between $\{1, 2\}$ and $\{6, 8\}$? **Answer:** 4, 7, 5.5**.**

33. Number of merges to build a full dendrogram of n points? **Answer:** $n - 1$**.**

34. $C = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$: eigenvalues and variance explained by PC1? **Answer:** $3, 1$**;** 75%**.**

35. Relationship of the SVD singular values σ_j of the centred data matrix to PCA eigenvalues? **Answer:** $\lambda_j = \sigma_j^2/(n-1)$**.**

How to study this section: Highest-yield for GATE DA are small numeric problems: least-squares and ridge by hand, one logistic or naive-Bayes computation, entropy/information gain, SVM margin ($2/\|\boldsymbol{w}\|$) and support vectors, forward pass and parameter counts of a network, bias–variance diagnosis, LOOCV and k-fold counts, a k-means iteration, linkage distances in hierarchical clustering, and eigenvalues of a 2×2 covariance matrix for PCA. Practise each by hand until it takes under three minutes.